Волгоградский государственный университет
Институт математики и информационных технологий
Кафедра информационных систем и компьютерного моделирования

Работа допущена к защите
Заведующий кафедрой ИСКМ

_____ А. В. Хоперсков

«____» _____ 2026 г.

Богунь Марк Витальевич

Параллельная OpenMP-версия программы для численного моделирования теплопередачи в сплошных средах

Отчет по производственной практике, научно-исследовательской работе
(седьмой семестр)

Направление: 09.03.04 Программная инженерия

Студент группы ПРИб-231:

Богунь Марк Витальевич _____
подпись

Руководитель практики:

Двужилова Ю. В. — указать ФИО полностью, д.ф.-м.н., доцент каф. ИСКМ

_____
подпись

Волгоград — 2026

Содержание

Введение 5

Глава 1. Параллельные вычисления в задаче моделирования теплопередачи 8

1.1 Математическая модель нестационарной теплопроводности в трёхмерной области 8

1.1.1 Уравнение теплопроводности с граничными условиями первого рода 8

1.1.2 Явная конечно-разностная схема и условие её устойчивости 9

1.2 Технология OpenMP для вычислительных систем с общей памятью 10

1.2.1 Модель исполнения fork–join и директивы распараллеливания циклов 10

1.2.2 Распределение итераций между потоками и редукция данных 11

1.3 Показатели эффективности параллельных программ и их ограничения 12

1.3.1 Ускорение и эффективность параллельной программы на многоядерном процессоре 12

1.3.2 Закон Амдала и ограничение пропускной способностью памяти 12

Глава 2. Разработка параллельной OpenMP-версии программного комплекса HeatSim3D 14

2.1 Анализ информационных зависимостей вычислительного алгоритма схемы FTCS 14

2.1.1 Независимость вычислений во внутренних узлах на одном шаге . . . 14
2.1.2 Организация двух буферов температурного поля в памяти . . . 15
2.2 Распараллеливание основных вычислительных циклов расчётного модуля . . . 15
2.2.1 Слияние пространственных циклов для равномерной загрузки потоков . . . 15
2.2.2 Параллельное наложение граничных условий и вычисление средней температуры . . . 16
2.2.3 Управление числом потоков и измерение времени расчёта 17
2.3 Обеспечение кроссплатформенности программного комплекса для Linux и Windows . . . 17
2.3.1 Сборка расчётного модуля различными компиляторами с поддержкой OpenMP . . . 17
2.3.2 Средства построения графиков и автоматизации вычислительных экспериментов . . . 18

Глава 3. Вычислительные эксперименты с параллельной версией программного комплекса . . . 20
3.1 Проверка корректности результатов параллельной версии программы . . . 20
3.1.1 Сравнение численного решения с аналитическим решением задачи . . . 20
3.1.2 Исследование сходимости численного решения при измельчении сетки . . . 21
3.1.3 Совпадение результатов при различном числе вычислительных потоков . . . 22

3.2 Исследование производительности параллельной версии на многоядерном процессоре 23
3.2.1 Методика проведения замеров времени выполнения расчёта 23
3.2.2 Ускорение и эффективность для расчётных сеток разного размера 24
3.2.3 Влияние последовательной записи результатов на итоговое ускорение 25

Заключение 28

Список литературы 31

Приложение А. Фрагменты исходного кода параллельного расчётного модуля 33

Приложение Б. Сборка программного комплекса и запуск вычислительных экспериментов 35

Ход выполнения практики 36

Введение

Численное моделирование теплопередачи широко применяется при проектировании изделий машиностроения, энергетики и электроники, где температурный режим определяет надёжность конструкции [1]. Описание процесса строится на уравнении теплопроводности Фурье–Кирхгофа [2], которое для тел сложной формы и при неоднородных граничных условиях, как правило, не имеет аналитического решения. Поэтому на практике используют сеточные методы, в которых расчётная область заменяется множеством узлов, а производные — конечными разностями [3].

В предыдущем семестре был разработан программный комплекс HeatSim3D, который решает трёхмерную нестационарную задачу теплопроводности по явной конечно-разностной схеме FTCS. Комплекс состоит из расчётного модуля на языке C++ [4] и графического интерфейса на языке Python [5]. Работа с ним показала, что время расчёта быстро растёт при измельчении сетки. При уменьшении шага сетки вдвое число узлов увеличивается в восемь раз, а допустимый шаг по времени из условия устойчивости уменьшается в четыре раза, так что общий объём вычислений возрастает примерно в тридцать два раза. Для сетки из нескольких миллионов узлов один расчёт в последовательном режиме занимает уже минуты, а серия расчётов для разных материалов и граничных условий — часы.

Современные персональные компьютеры оснащены многоядерными процессорами, однако последовательная программа использует только одно ядро. Наиболее простой способ задействовать все ядра в программе на C++ даёт технология OpenMP [6], которая позволяет распараллелить существующий код добавлением директив компилятора без изменения структуры алгоритма [7]. При этом важно не только получить выигрыш во времени, но и доказать, что параллельная программа выдаёт те же результаты, что и последовательная, а также оценить, насколько полно используются вычислитель-

ные ресурсы. Именно количественная оценка ускорения и анализ факторов, которые его ограничивают, составляют исследовательскую часть работы.

Целью работы является разработка и исследование параллельной OpenMP-версии программного комплекса HeatSim3D для численного моделирования нестационарной теплопередачи в сплошных средах.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Изучить технологию OpenMP и показатели эффективности параллельных программ для систем с общей памятью.

2. Проанализировать информационные зависимости алгоритма явной схемы FTCS и выделить участки, допускающие параллельное выполнение.

3. Разработать параллельную версию расчётного модуля с управляемым числом потоков, совместимую с компиляторами GCC, MinGW и Microsoft Visual C++.

4. Обеспечить сборку и работу программного комплекса в операционных системах Linux и Windows.

5. Проверить корректность параллельной версии путём сравнения с аналитическим решением и с результатами последовательного расчёта.

6. Исследовать ускорение и эффективность параллельной версии для расчётных сеток разного размера и оценить влияние последовательной записи результатов.

Новизна работы для программного комплекса HeatSim3D состоит в том, что параллельный режим впервые снабжён средствами управления числом потоков и автоматического замера производительности. Это позволило перейти от качественного утверждения об ускорении расчёта к измеренным значениям ускорения и эффективности. Кроме того, разработан модуль построения графиков в оформлении, пригодном для научного отчёта, и набор автоматических тестов, подтверждающих корректность параллельной версии.

В первой главе приводится краткое описание математической модели и

численного метода, рассматриваются модель исполнения OpenMP и показатели эффективности параллельных программ. Во второй главе описана разработка параллельной версии расчётного модуля, включая анализ зависимостей по данным, выбор директив OpenMP и обеспечение кроссплатформенности. В третьей главе представлены результаты проверки корректности и исследования производительности параллельной версии на многоядерном процессоре.

Глава 1

Параллельные вычисления в задаче моделирования теплопередачи

1.1 Математическая модель нестационарной теплопроводности в трёхмерной области

1.1.1 Уравнение теплопроводности с граничными условиями первого рода

Рассматривается однородное изотропное твёрдое тело в форме куба $\Omega = [0, L]^3$ с длиной ребра $L = 1$ м. Распределение температуры $T(x, y, z, t)$ в нём при отсутствии внутренних источников тепла описывается уравнением теплопроводности Фурье–Кирхгофа [8]

$$\frac{\partial T}{\partial t} = a\left(\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} + \frac{\partial^2 T}{\partial z^2}\right), \tag{1.1.1}$$

где a — коэффициент температуропроводности материала, м^2/с. Этот коэффициент равен отношению теплопроводности к произведению плотности на удельную теплоёмкость и характеризует скорость выравнивания температуры в теле [9]. Для конструкционной стали, которая используется в расчётах данной работы, принято значение $a = 1{,}28 \cdot 10^{-5}$ м^2/с.

Для однозначного решения уравнения (1.1.1) задаются начальное и граничные условия [10]. В начальный момент температура во всех точках тела одинакова и равна T_0. На каждой из шести граней куба поддерживается собственная постоянная температура, то есть используются граничные условия первого рода, или условия Дирихле [11]. Например, на гранях, перпендикулярных оси x, они имеют вид

$$T\big|_{x=0} = T_{x-}, \qquad T\big|_{x=L} = T_{x+}, \tag{1.1.2}$$

а на остальных гранях записываются аналогично. Независимое задание температуры на каждой грани позволяет моделировать как равномерный прогрев тела, так и направленный тепловой поток. Подробный вывод уравнения и обсуждение физического смысла его параметров приведены в отчёте за предыдущий семестр, поэтому здесь математическая модель изложена кратко.

1.1.2 Явная конечно-разностная схема и условие её устойчивости

Расчётная область покрывается равномерной сеткой из $N_x \times N_y \times N_z$ узлов с шагами $h_x = L/(N_x - 1)$, h_y и h_z, которые определяются аналогично. Обозначим через $T^n_{i,j,k}$ приближённое значение температуры в узле с индексами (i, j, k) в момент времени $t_n = n\Delta t$. Замена производной по времени разностью вперёд, а вторых производных по пространству центральными разностями приводит к явной схеме FTCS [3]

$$T^{n+1}_{i,j,k} = T^n_{i,j,k} + r_x\,\delta^2_x T^n_{i,j,k} + r_y\,\delta^2_y T^n_{i,j,k} + r_z\,\delta^2_z T^n_{i,j,k}, \tag{1.1.3}$$

в которой использованы обозначения $\delta^2_x T_{i,j,k} = T_{i+1,j,k} - 2T_{i,j,k} + T_{i-1,j,k}$ и $r_x = a\Delta t/h_x^2$, а величины для осей y и z определяются так же. Схема (1.1.3) имеет первый порядок аппроксимации по времени и второй порядок по пространству.

Явная схема устойчива только при ограничении на шаг по времени [12]. Для трёхмерной задачи оно записывается в виде неравенства

$$2\,(r_x + r_y + r_z) \le 1, \tag{1.1.4}$$

поэтому в программе шаг Δt выбирается автоматически так, чтобы левая часть (1.1.4) равнялась 0,4. Такой запас исключает рост ошибок округления при любых допустимых параметрах расчёта. Из неравенства (1.1.4) следует, что шаг по времени пропорционален квадрату шага сетки, и именно это делает расчёт на мелких сетках трудоёмким.

Для дальнейшего важно одно свойство схемы (1.1.3). Новое значение температуры в узле вычисляется только по значениям с предыдущего временного слоя в самом узле и в шести соседних узлах. Следовательно, на одном шаге по времени все внутренние узлы можно обрабатывать независимо друг от друга и в произвольном порядке, что и создаёт возможность для параллельных вычислений.

1.2 Технология OpenMP для вычислительных систем с общей памятью

1.2.1 Модель исполнения fork–join и директивы распараллеливания циклов

OpenMP представляет собой стандарт программного интерфейса для параллельного программирования на языках C, C++ и Fortran в системах с общей памятью [6]. Стандарт включает директивы компилятора, библиотечные функции и переменные окружения[1]. Программа начинает работу в одном главном потоке, а при входе в параллельную область создаётся группа потоков, которые выполняют её совместно. По завершении области потоки синхронизируются, и выполнение продолжается в главном потоке. Такая схема называется моделью fork–join [7].

В вычислительных программах основная работа сосредоточена в циклах, поэтому наиболее употребительна директива `parallel for`. Она распределяет итерации следующего за ней цикла между потоками группы, причём программисту не требуется вручную создавать потоки и делить диапазон индексов [13]. Все потоки работают в общем адресном пространстве, поэтому массивы доступны каждому из них без копирования. Обратной стороной общей памяти является риск гонки данных, при которой несколько потоков

[1] Актуальные версии спецификации OpenMP публикуются на официальном сайте OpenMP Architecture Review Board: `https://www.openmp.org`.

одновременно изменяют одну и ту же ячейку и результат зависит от порядка их выполнения. Если программа компилируется без поддержки OpenMP, директивы игнорируются и получается обычная последовательная программа, что удобно для сравнения двух версий.

1.2.2 Распределение итераций между потоками и редукция данных

Способ распределения итераций задаётся параметром `schedule`. При статическом распределении диапазон итераций заранее делится на равные непрерывные части по числу потоков, а при динамическом распределении потоки получают новые порции итераций по мере завершения предыдущих [13]. Статическое распределение не требует дополнительной синхронизации во время работы цикла и поэтому предпочтительно, когда все итерации выполняют одинаковый объём вычислений. Для вложенных циклов стандарт, начиная с версии 3.0, предоставляет параметр `collapse`, объединяющий несколько циклов в одно пространство итераций.

Отдельного внимания требуют циклы, в которых все итерации накапливают результат в одной переменной, например при суммировании элементов массива. Параметр `reduction` создаёт для каждого потока собственную копию такой переменной, а по окончании цикла объединяет частичные результаты [7]. Этот приём устраняет гонку данных без использования блокировок. Однако порядок сложения при этом меняется, и для чисел с плавающей точкой результат может отличаться от последовательного в последних значащих разрядах.

1.3 Показатели эффективности параллельных программ и их ограничения

1.3.1 Ускорение и эффективность параллельной программы на многоядерном процессоре

Качество распараллеливания оценивается по времени выполнения одной и той же задачи на разном числе потоков. Если t_1 — время расчёта одним потоком, а t_p — время расчёта p потоками, то ускорение и эффективность определяются выражениями

$$S_p = \frac{t_1}{t_p}, \qquad E_p = \frac{S_p}{p}. \tag{1.3.1}$$

В идеальном случае ускорение равно числу потоков, а эффективность равна единице. На практике эффективность меньше единицы из-за затрат на создание и синхронизацию потоков, неравномерной загрузки и наличия участков программы, которые выполняются последовательно. Значения эффективности выше 0,8 обычно считаются хорошим результатом для программ с общей памятью.

1.3.2 Закон Амдала и ограничение пропускной способностью памяти

Пусть доля f времени работы последовательной программы приходится на участки, которые не распараллеливаются. Тогда по закону Амдала [14] ускорение на p потоках не может превысить величину

$$S_p = \frac{1}{f + (1 - f)/p}, \tag{1.3.2}$$

и при неограниченном увеличении числа потоков стремится к $1/f$. Например, при $f = 0{,}1$ ускорение на четырёх потоках не превышает 3,1, а на любом числе потоков — 10. В программе моделирования теплопередачи последовательной

частью является прежде всего запись промежуточных результатов в файлы, поскольку она выполняется одним потоком.

Второе ограничение связано с работой памяти. При вычислении нового значения по формуле (1.1.3) на каждый узел приходится около десяти арифметических операций и обращение к семи значениям массива. Процессор выполняет такие операции быстрее, чем память успевает доставлять данные, поэтому скорость расчёта определяется пропускной способностью памяти. Ядра процессора разделяют общую шину памяти и кэш последнего уровня [15], и с ростом числа потоков каждое ядро получает меньшую долю этой пропускной способности. По этой причине для задач со стенсильными вычислениями эффективность обычно снижается с ростом числа потоков даже при отсутствии последовательных участков.

Глава 2

Разработка параллельной OpenMP-версии программного комплекса HeatSim3D

2.1 Анализ информационных зависимостей вычислительного алгоритма схемы FTCS

2.1.1 Независимость вычислений во внутренних узлах на одном шаге

Расчётный модуль HeatSim3D принимает параметры задачи через аргументы командной строки, выполняет цикл по времени и сохраняет результаты в файлы формата CSV. Каждый шаг по времени состоит из трёх действий. Сначала по формуле (1.1.3) вычисляются новые значения во всех внутренних узлах, затем на шести гранях восстанавливаются граничные значения, и, наконец, с заданной периодичностью в файлы записываются срезы поля и температура в центре тела.

Первое действие занимает основную часть времени расчёта, так как число внутренних узлов равно $(N_x - 2)(N_y - 2)(N_z - 2)$ и для сетки 150^3 превышает три миллиона. Как отмечено в первой главе, значение в каждом узле нового слоя зависит только от значений предыдущего слоя. Поэтому между итерациями цикла по внутренним узлам отсутствуют зависимости по данным, и цикл может выполняться параллельно без изменения результата. Между шагами по времени, напротив, существует строгая зависимость, поскольку следующий шаг использует результат предыдущего. Таким образом, распараллеливанию подлежит обработка узлов внутри шага, а сам цикл по времени остаётся последовательным.

2.1.2 Организация двух буферов температурного поля в памяти

Температурное поле хранится в одномерном массиве `std::vector<double>` длиной $N_x N_y N_z$, а узлу с индексами (i, j, k) соответствует элемент с номером $iN_yN_z + jN_z + k$. При такой нумерации соседние по индексу k узлы расположены в памяти подряд, что позволяет процессору эффективно использовать кэш при последовательном проходе по внутреннему циклу.

Для корректной параллельной работы используются два массива одинакового размера. Первый массив содержит значения на текущем слое и во время шага только читается, а во второй массив записываются новые значения. После завершения шага массивы меняются ролями с помощью функции `std::swap`, которая обменивает внутренние указатели и не копирует данные. При такой организации каждый элемент нового массива записывается ровно одним потоком, а читаемый массив не изменяется, поэтому гонка данных невозможна. Синхронизация требуется только перед обменом массивов, и её обеспечивает неявный барьер в конце директивы `parallel for`.

2.2 Распараллеливание основных вычислительных циклов расчётного модуля

2.2.1 Слияние пространственных циклов для равномерной загрузки потоков

В исходной версии программы цикл по внутренним узлам состоял из трёх вложенных циклов по индексам i, j и k. Если распараллелить только внешний цикл, то число его итераций $N_x - 2$ при небольших сетках сравнимо с числом потоков, и часть потоков может получить на одну итерацию больше других. В версии прошлого семестра для объединения двух внешних циклов использовался параметр `collapse(2)`. Однако этот параметр появился толь-

ко в стандарте OpenMP 3.0, а компилятор Microsoft Visual C++ с ключом `/openmp` поддерживает стандарт 2.0, и программа в этой среде не собиралась бы.

Поэтому в новой версии слияние циклов выполнено вручную. Вводится единый индекс m, пробегающий значения от 0 до $(N_x - 2)(N_y - 2) - 1$, а исходные индексы восстанавливаются по формулам

$$i = 1 + \left\lfloor \frac{m}{N_y - 2} \right\rfloor, \qquad j = 1 + m \bmod (N_y - 2), \tag{2.2.1}$$

после чего внутри итерации последовательно обрабатывается весь столбец узлов по индексу k. Директива `parallel for` со статическим распределением применяется к циклу по m. Уже для сетки 50^3 число итераций равно 2304, поэтому части, достающиеся потокам, практически одинаковы. Внутренний цикл по k проходит по памяти подряд и хорошо оптимизируется компилятором. Фрагмент исходного кода основного цикла приведён в Приложении А.

2.2.2 Параллельное наложение граничных условий и вычисление средней температуры

Наложение граничных условий выполняется после каждого шага и состоит из трёх пар циклов, каждая из которых заполняет две противоположные грани куба. Объём этой работы пропорционален N^2, а не N^3, однако при последовательном выполнении она превращалась бы в постоянную последовательную долю каждого шага. Поэтому каждая пара циклов также снабжена директивой `parallel for`. Записи в разные элементы массива не пересекаются, и синхронизация внутри этих циклов не нужна.

При каждой записи результатов программа вычисляет среднюю температуру по объёму тела, то есть сумму значений во всех узлах. Для этого цикла использована редукция `reduction(+:sum)`, при которой каждый поток суммирует свою часть массива, а частичные суммы объединяются по окончании цикла [7]. Из-за изменения порядка сложения средняя температура может от-

личаться от последовательного значения на величину порядка 10^{-13} °C. Такое отличие не проявляется в файлах результатов, где значения записываются с четырьмя знаками после запятой.

2.2.3 Управление числом потоков и измерение времени расчёта

Для исследования производительности в расчётный модуль добавлен параметр командной строки `-threads`, который вызывает функцию `omp_set_num_threads`. Если параметр не указан, используется число потоков, выбранное средой OpenMP по умолчанию, обычно равное числу ядер процессора. Фактическое число потоков выводится в заголовке протокола расчёта и сохраняется вместе со всеми параметрами задачи в файл `params.csv`. Это позволяет по файлам результатов однозначно установить, в каком режиме был выполнен расчёт.

Время расчёта измеряется средствами библиотеки `std::chrono` от начала до конца цикла по времени, поэтому выделение памяти и разбор параметров в него не входят. Измеренное время выводится по окончании расчёта и также записывается в `params.csv`. При сборке без поддержки OpenMP получается последовательная версия программы, в которой параметр `-threads` не действует, а в протоколе указывается один поток.

2.3 Обеспечение кроссплатформенности программного комплекса для Linux и Windows

2.3.1 Сборка расчётного модуля различными компиляторами с поддержкой OpenMP

Исходный код расчётного модуля написан на стандартном языке C++17 и содержит единственный платформенно-зависимый фрагмент, связанный с созданием каталога для результатов. В операционной системе Windows для

этого используется функция `_mkdir`, а в Linux — функция `mkdir`, и выбор между ними выполняется директивами условной компиляции. Обращения к библиотеке OpenMP также помещены в блоки условной компиляции, поэтому один и тот же файл собирается как в параллельном, так и в последовательном режиме.

Для Linux расчётный модуль собирается компилятором GCC с ключом `-fopenmp`. Для Windows исполняемый файл получен кросс-компилятором MinGW-w64 на основе GCC 13 со статической компоновкой, благодаря чему он не требует установки дополнительных библиотек на компьютере пользователя. Сборка в среде Microsoft Visual C++ также возможна после включения поддержки OpenMP в свойствах проекта. Графический интерфейс автоматически выбирает исполняемый файл расчётного модуля в зависимости от операционной системы, а сценарии сборки для обеих систем, включая упаковку интерфейса в один исполняемый файл, приведены в Приложении Б.

2.3.2 Средства построения графиков и автоматизации вычислительных экспериментов

Для подготовки результатов к отчёту разработан модуль `report_plots.py` на языке Python, использующий библиотеку matplotlib [16]. Модуль строит графики в едином оформлении, в котором кривые различаются типом линии и маркерами, а шрифт на рисунках совпадает со шрифтом основного текста по начертанию и размеру. Помимо графиков динамики температуры и профилей, модуль вычисляет аналитическое решение для проверки модели и выполняет серию запусков расчётного модуля с разным числом потоков для оценки ускорения.

Все вычислительные эксперименты, результаты которых приведены в третьей главе, выполняются одним сценарием `experiments.py`. Сценарий запускает расчёты, сохраняет исходные данные замеров в файлы CSV и строит по ним рисунки. Такая организация делает результаты воспроизводимыми,

поскольку любой рисунок отчёта можно получить повторно на другом компьютере одной командой. Корректность параллельной версии дополнительно контролируется набором автоматических тестов на основе библиотеки pytest, которые проверяют совпадение результатов при разном числе потоков, работу параметра `-threads` и соответствие аналитическому решению.

Глава 3

Вычислительные эксперименты с параллельной версией программного комплекса

3.1 Проверка корректности результатов параллельной версии программы

3.1.1 Сравнение численного решения с аналитическим решением задачи

Корректность численной модели проверялась на задаче, для которой известно точное решение. Рассматривался прогрев стального куба с начальной температурой $T_0 = 20$ °C, на всех гранях которого поддерживается температура $T_b = 100$ °C. Методом разделения переменных решение такой задачи представляется рядом Фурье, и в центре куба оно принимает вид

$$T_c(t) = T_b + (T_0 - T_b)\,\frac{64}{\pi^3}\left(\sum_{k=1,3,5,\ldots}\frac{(-1)^{(k-1)/2}}{k}\,e^{-a\pi^2k^2t/L^2}\right)^3, \qquad (3.1.1)$$

где суммирование выполняется по нечётным значениям k. Ряд в выражении (3.1.1) распадается на произведение трёх одинаковых одномерных рядов, поэтому его легко вычислить с высокой точностью. В расчётах использовались первые двадцать нечётных членов ряда, чего достаточно при $t > 0$. Численный расчёт выполнялся параллельной версией программы на сетке 31^3 с шагом по времени 5,79 с до момента $t = 20\,000$ с.

На рисунке 3.1 показано изменение температуры в центре куба, полученное численно и по формуле (3.1.1), а также модуль их разности. Кривые на рисунке 3.1, а практически совпадают на всём интервале времени: после короткого начального периода, пока тепло от граней ещё не дошло до центра, температура быстро растёт и затем асимптотически приближается к 100 °C.

График погрешности на рисунке 3.1, б имеет один максимум, равный 0,20 °C, который достигается примерно через 0,55 ч после начала прогрева, когда температура меняется быстрее всего. Это значение составляет 0,25 % от полного перепада температур 80 °C. По мере выхода на установившийся режим погрешность уменьшается и стремится к нулю, что соответствует поведению устойчивой разностной схемы.

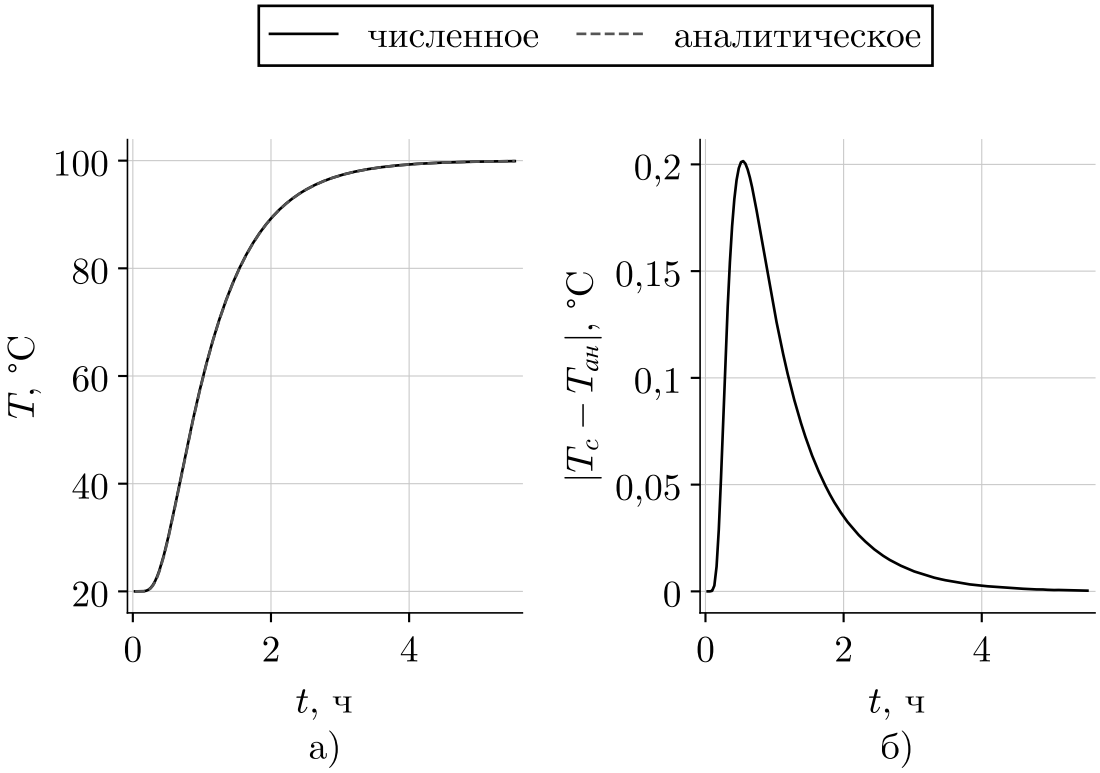


Рисунок 3.1 — Температура в центре куба: численное и аналитическое решения (а) и абсолютная погрешность численного решения (б)

3.1.2 Исследование сходимости численного решения при измельчении сетки

Совпадение с аналитическим решением на одной сетке ещё не доказывает правильность реализации, поскольку ошибка в программе могла бы проявиться только в порядке сходимости. Поэтому расчёт той же задачи был повторён на четырёх сетках, шаг которых последовательно уменьшался вдвое.

Шаг по времени при этом выбирался программой автоматически из условия (1.1.4). Погрешность сравнивалась в момент $t = 4000$ с, а численное значение в этот момент находилось линейной интерполяцией между ближайшими шагами по времени. Результаты сравнения приведены в таблице 3.1.

Таблица 3.1 — Погрешность температуры в центре куба при $t = 4000$ с

Сетка	h, м	T_c, °C	$\lvert T_c - T_{\text{ан}}\rvert$, °C	Отношение погрешностей
11^3	0,1000	65,383	1,0113	—
21^3	0,0500	64,626	0,2543	3,98
41^3	0,0250	64,435	0,0636	4,00
81^3	0,0125	64,387	0,0159	4,00

Аналитическое значение температуры в этот момент равно 64,371 °C. Из таблицы видно, что при каждом уменьшении шага сетки вдвое погрешность уменьшается ровно в четыре раза. Такое поведение означает второй порядок сходимости по пространству, который и должна обеспечивать схема (1.1.3). Первый порядок аппроксимации по времени не нарушает эту картину, так как шаг по времени пропорционален h^2 и вносимая им ошибка убывает так же быстро.

3.1.3 Совпадение результатов при различном числе вычислительных потоков

Для параллельной программы недостаточно проверить точность одного расчёта, так как ошибки синхронизации проявляются случайно и зависят от числа потоков. Поэтому одна и та же задача с несимметричными граничными условиями на сетке 21^3 рассчитывалась одним и четырьмя потоками, после чего все файлы результатов сравнивались поэлементно. Значения температуры во всех срезах поля совпали побитно, то есть без какого-либо различия даже в последнем разряде. Такой результат объясняется тем, что каждый

узел вычисляется по одной и той же формуле с одними и теми же исходными данными, а от числа потоков зависит только то, какой поток выполняет вычисление.

Аналогичное сравнение было выполнено для исполняемых файлов, собранных для Linux и для Windows. Версия для Windows запускалась в среде совместимости Wine, и её результаты также совпали с результатами версии для Linux побитно, а различие файлов состояло лишь в символах перевода строки. Оба сравнения включены в набор автоматических тестов и выполняются при каждом изменении исходного кода.

3.2 Исследование производительности параллельной версии на многоядерном процессоре

3.2.1 Методика проведения замеров времени выполнения расчёта

Замеры выполнялись на четырёхъядерном процессоре Intel Xeon с тактовой частотой 2,1 ГГц под управлением операционной системы Linux. Расчётный модуль был собран компилятором GCC версии 13.3 с уровнем оптимизации `-O2`. Рассматривались кубические сетки 50^3, 100^3 и 150^3, содержащие от 125 тысяч до 3,4 миллиона узлов, а число потоков менялось от одного до четырёх.

Чтобы время расчёта на разных сетках было сопоставимым, на каждой сетке выполнялась ровно тысяча шагов по времени, а запись промежуточных результатов была отключена. Каждый расчёт повторялся пять раз, и в качестве результата принималось наименьшее время. Такой выбор исключает случайные задержки, вызванные другими процессами операционной системы, и характеризует возможности самой программы. Ускорение и эффективность вычислялись по формулам (1.3.1).

3.2.2 Ускорение и эффективность для расчётных сеток разного размера

Измеренное время расчёта приведено в таблице 3.2. Последовательная версия программы, собранная без поддержки OpenMP, на сетке 100^3 выполняет расчёт за 2,02 с, то есть практически за то же время, что и параллельная версия на одном потоке. Это означает, что сами директивы OpenMP не создают заметных накладных расходов.

Таблица 3.2 — Время расчёта 1000 шагов по времени, с

Сетка	$p = 1$	$p = 2$	$p = 3$	$p = 4$	S_4	E_4
50^3	0,231	0,132	0,107	0,085	2,72	0,68
100^3	1,879	1,010	0,737	0,548	3,43	0,86
150^3	6,837	3,520	2,565	2,121	3,22	0,81

Зависимость ускорения и эффективности от числа потоков для трёх сеток показана на рисунке 3.2. На рисунке 3.2, а кривые для сеток 100^3 и 150^3 идут вблизи идеальной прямой $S_p = p$ и при четырёх потоках достигают значений 3,43 и 3,22 соответственно. Кривая для сетки 50^3 заметно отклоняется от идеальной уже при трёх потоках, и итоговое ускорение составляет 2,72. На рисунке 3.2, б эта разница видна ещё отчётливее: эффективность для больших сеток при четырёх потоках остаётся на уровне 0,81–0,86, тогда как для малой сетки она снижается до 0,68.

Снижение эффективности на малой сетке объясняется соотношением полезной работы и накладных расходов. На сетке 50^3 один шаг по времени выполняется за доли миллисекунды, а на каждом шаге программа проходит через четыре параллельные области, в конце каждой из которых потоки синхронизируются. Время синхронизации от размера сетки не зависит, поэтому на малой сетке его доля оказывается значительной. На больших сетках доля синхронизации мала, и основным ограничением становится пропускная

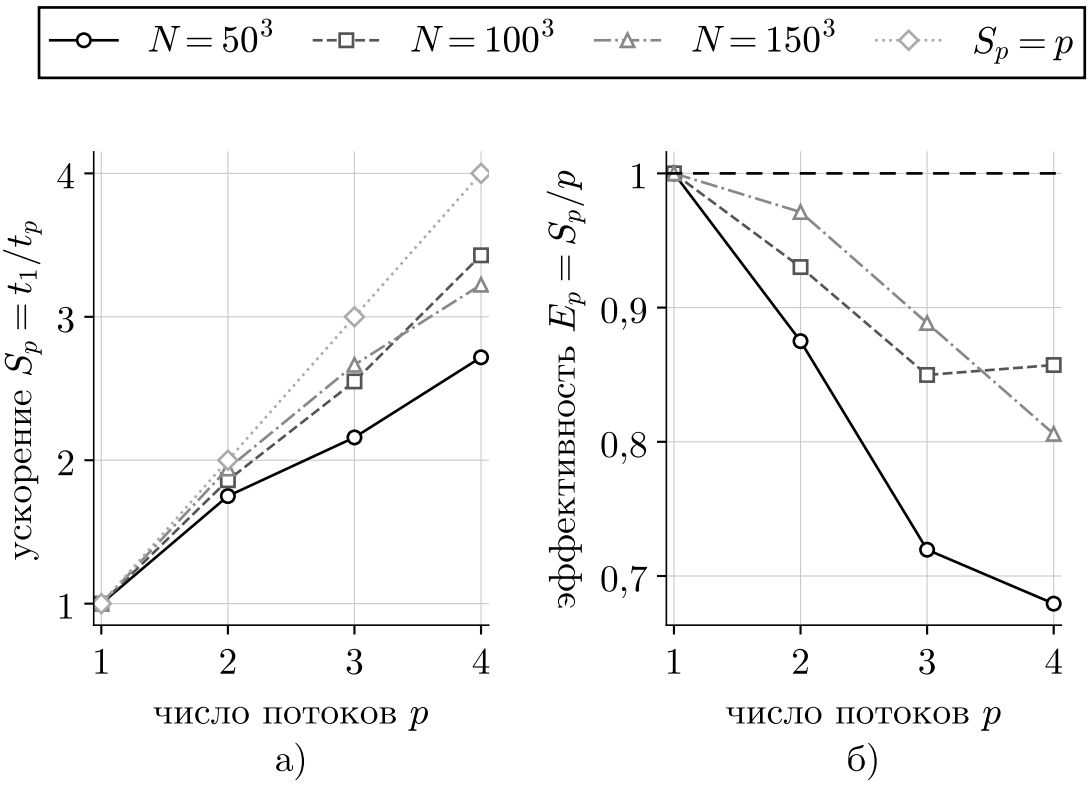


Рисунок 3.2 — Ускорение (а) и эффективность (б) параллельной версии программы в зависимости от числа потоков для сеток разного размера

способность памяти, о которой говорилось в первой главе. Различие между сетками 100^3 и 150^3 находится в пределах разброса повторных замеров, составляющего около 5–10 %.

3.2.3 Влияние последовательной записи результатов на итоговое ускорение

В рассмотренных замерах запись результатов была отключена, однако в реальной работе программа периодически сохраняет срезы температурного поля для последующей визуализации. Запись в файлы выполняется одним потоком, и для оценки её влияния был проведён отдельный эксперимент на сетке 100^3. При одной и той же тысяче шагов число записей M менялось от нуля до ста, а время расчёта измерялось на одном и на четырёх потоках. Для сравнения с замерами использовалась простая модель, в которой время

записи не уменьшается при увеличении числа потоков, а время вычислений уменьшается так же, как в расчёте без записи.

Результаты эксперимента приведены на рисунке 3.3. На рисунке 3.3, а время расчёта на обоих потоках растёт с увеличением числа записей, причём для одного потока каждая запись добавляет около 17 мс. Поскольку это время одинаково для любого числа потоков, разрыв между кривыми в относительном выражении сокращается. На рисунке 3.3, б видно, что ускорение на четырёх потоках падает с 3,39 при отсутствии записи до 2,16 при двадцати записях и до 1,34 при ста записях.

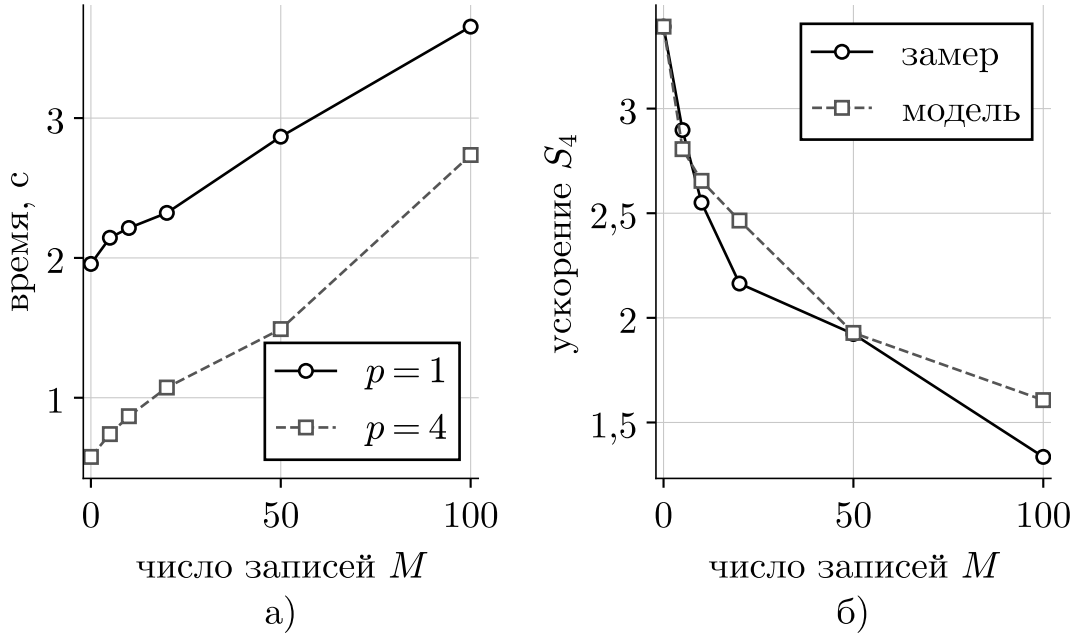


Рисунок 3.3 — Время расчёта (а) и ускорение на четырёх потоках (б) в зависимости от числа записей результатов на сетке 100^3

Модельная кривая хорошо описывает замеры при небольшом числе записей, что подтверждает применимость закона Амдала (1.3.2) к данной программе. При ста записях на запись приходится 0,46 времени последовательного расчёта, и модель предсказывает ускорение не выше 1,6. Фактическое ускорение оказывается ещё ниже модельного, вероятно, из-за того, что во время записи остальные потоки простаивают в ожидании, а данные расчётного массива вытесняются из кэша процессора. Отсюда следует практическая

рекомендация: при длительных расчётах интервал записи следует выбирать так, чтобы число сохранённых срезов составляло десятки, а не сотни. Дальнейшее повышение эффективности возможно за счёт выполнения записи в отдельном потоке одновременно с вычислением следующих шагов.

Заключение

В ходе работы разработана и исследована параллельная версия программного комплекса HeatSim3D, предназначенного для численного моделирования нестационарной теплопередачи в трёхмерных сплошных средах. Распараллеливание выполнено средствами технологии OpenMP для многоядерных процессоров с общей памятью. Работа опирается на программный комплекс, созданный в предыдущем семестре, и сохраняет его математическую модель, численный метод и графический интерфейс.

В первой главе кратко изложена математическая модель на основе уравнения Фурье–Кирхгофа с граничными условиями первого рода на шести гранях и явная конечно-разностная схема FTCS с условием устойчивости. Рассмотрены модель исполнения OpenMP, способы распределения итераций между потоками и механизм редукции. Показано, что ускорение параллельной программы ограничивается последовательными участками в соответствии с законом Амдала, а для стенсильных вычислений — также пропускной способностью памяти.

Во второй главе на основе анализа информационных зависимостей установлено, что все внутренние узлы на одном шаге по времени можно обрабатывать независимо, если хранить текущий и новый слои температурного поля в разных массивах. Распараллелены основной цикл схемы, наложение граничных условий и вычисление средней температуры. Ручное слияние двух внешних циклов обеспечило равномерную загрузку потоков и совместимость со стандартом OpenMP 2.0, который поддерживается компилятором Microsoft Visual C++. В расчётный модуль добавлено управление числом потоков, а программный комплекс собирается и работает в операционных системах Linux и Windows.

В третьей главе подтверждена корректность параллельной версии. Максимальное отклонение численного решения от аналитического на сетке 31^3

составило 0,20 °C, или 0,25 % от перепада температур, а при измельчении сетки погрешность уменьшается в четыре раза при каждом уменьшении шага вдвое, что соответствует второму порядку сходимости. Результаты расчёта при одном и четырёх потоках, а также в версиях для Linux и Windows совпадают побитно. Исследование производительности на четырёхъядерном процессоре показало, что для сеток от миллиона узлов ускорение на четырёх потоках составляет 3,2–3,4 раза при эффективности 0,81–0,86. На малой сетке 50^3 эффективность снижается до 0,68 из-за относительно большой доли затрат на синхронизацию потоков. Частая последовательная запись результатов способна снизить ускорение почти втрое, что согласуется с законом Амдала и позволяет дать практическую рекомендацию по выбору интервала сохранения.

Таким образом, цель работы достигнута, и параллельная версия позволяет выполнять расчёты на мелких сетках в несколько раз быстрее при неизменной точности. Направлениями дальнейшего развития являются асинхронная запись результатов в отдельном потоке, блочная обработка расчётной области для лучшего использования кэша процессора и исследование масштабируемости на процессорах с большим числом ядер.

Перечень компетенций ниже повторяет список прошлого семестра. Обязательно сверить с учебным планом седьмого семестра: при несоответствии работа снимается с защиты.

Перечень компетенций для направления 09.03.04 «Программная инженерия», сформированных в ходе производственной практики (научно-исследовательской работы):

УК-1. Способен осуществлять поиск, критический анализ и синтез информации, применять системный подход для решения задач. Компетенция сформирована при изучении литературы по технологии OpenMP и показателям эффективности параллельных программ, а также при анализе ограничений, связанных с законом Амдала и пропускной способностью памяти.

УК-4. Способен осуществлять деловую коммуникацию в устной и письменной формах на государственном языке Российской Федерации и иностранном(ых) языке(ах). Компетенция проявилась при работе с англоязычными источниками по параллельному программированию, подготовке отчётной документации и обсуждении результатов с руководителем практики.

ПК-1. Способен проводить научно-исследовательские и опытно-конструкторские разработки. Компетенция сформирована при разработке параллельной версии расчётного модуля, планировании и проведении вычислительных экспериментов по оценке ускорения и эффективности и анализе их результатов.

ПК-4. Способен создавать и анализировать требования на разработку программно-информационных систем. Компетенция проявилась при формулировании требований к параллельной версии, включая совпадение результатов с последовательной версией, управление числом потоков и работу в операционных системах Linux и Windows.

ПК-5. Способен осуществлять концептуальное, функциональное и логическое проектирование программно-информационных систем. Компетенция проявилась при проектировании организации данных для параллельной обработки, выборе схемы распределения вычислений между потоками и разработке средств автоматизации вычислительных экспериментов.

Список литературы

1. *Крайнов А. Ю.* Основы теплопередачи. Теплопередача через слой вещества. — Томский государственный университет, 2016. — 48 с. — Учебное пособие.

2. Математическое моделирование усадочных процессов стали на основе уравнений Навье-Стокса и Фурье-Кирхгофа / А. Н. Чичко [и др.] // Литьё и металлургия. — 2013. — 1 (69). — С. 14—21.

3. *Колган В. П.* Конечно-разностная схема для расчета двухмерных разрывных решений нестационарной газовой динамики // Ученые записки ЦАГИ. — 1975. — Т. 6, № 1. — С. 9—14.

4. *Stroustrup B.* An overview of C++ // Proceedings of the 1986 SIGPLAN workshop on Object-oriented programming. — 1986. — P. 7–18.

5. *Python Software Foundation.* Python 3.9.7 documentation // Python Releases for Windows. — 2021. — Vol. 24. — P. 1–1571.

6. *Dagum L.*, *Menon R.* OpenMP: an industry standard API for shared-memory programming // IEEE Computational Science and Engineering. — 1998. — Vol. 5, no. 1. — P. 46–55. — DOI: 10.1109/99.660313.

7. Parallel Programming in OpenMP / R. Chandra [et al.]. — San Francisco, CA : Elsevier Science, 2001. — P. 230. — (High performance computing). — ISBN 9781558606715.

8. *Сергиенко Л. С.* Исследование системы, обобщающей классическое уравнение теплопередачи // iPolytech Journal. — 2004. — 3 (19). — С. 142—148.

9. *Власов А. Б.* Исследование коэффициента температуропроводности электроизоляционных материалов с помощью тепловизора // Известия

Российского государственного педагогического университета им. А. И. Герцена. — 2004. — Т. 4, № 8. — С. 162—167.

10. *Luo C.*, *Moghtaderi B.*, *Page A.* Effect of ground boundary and initial conditions on the thermal performance of buildings // Applied Thermal Engineering. — 2010. — Vol. 30, no. 16. — P. 2602–2609.

11. *Алиев Н. А.*, *Велиева С. Р.* Смешанная задача для одного уравнения с частными производными четвертого порядка // Вестник Дагестанского государственного университета. Серия 1: Естественные науки. — 2018. — Т. 33, № 3. — С. 62—66.

12. *Dubey R. K.* On the stability of Forward in Time and Centred in Space (FTCS) scheme for scalar hyperbolic equation // arXiv preprint arXiv:1503.02629. — 2015. — P. 3–5.

13. *Антонов А. С.* Параллельное программирование с использованием технологии OpenMP. — Москва : Издательство Московского университета, 2009. — 77 с. — Учебное пособие.

14. *Amdahl G. M.* Validity of the single processor approach to achieving large scale computing capabilities // Proceedings of the April 18–20, 1967, Spring Joint Computer Conference (AFIPS '67). — ACM, 1967. — P. 483–485. — DOI: 10.1145/1465482.1465560.

15. *Кудухов А.* Особенности кэширования данных в информационных системах // Наука, Техника, Инновации 2014. — 2014. — С. 226—232.

16. *Bisong E.* Matplotlib and seaborn // Building machine learning and deep learning models on google cloud platform: A comprehensive guide for beginners. — Springer, 2019. — P. 151–165.

Приложение А

Фрагменты исходного кода параллельного расчётного модуля

В листинге А.1 показана установка числа потоков по параметру командной строки, в листинге А.2 — основной цикл по времени со слиянием пространственных циклов, а в листинге А.3 — вычисление средней температуры с редукцией. Полный исходный код находится в файле `heat3d.cpp` репозитория программного комплекса.

Листинг А.1 — Установка числа потоков OpenMP

```
int nThreads = 1;
#ifdef _OPENMP
    if (p.threads > 0) omp_set_num_threads(p.threads);
    nThreads = omp_get_max_threads();
#endif
```

Листинг А.2 — Основной цикл по времени

```
const int NIJ = (p.nx - 2) * (p.ny - 2);
for (int step = 0; step <= NSTEPS; ++step) {
    if (step % p.save_every == 0) {
        /* write T_center, T_mean and slices to CSV */
    }
    if (step == NSTEPS) break;

    #pragma omp parallel for schedule(static)
    for (int ij = 0; ij < NIJ; ++ij) {
        const int i = 1 + ij / (p.ny - 2);
        const int j = 1 + ij % (p.ny - 2);
        for (int k = 1; k < p.nz - 1; ++k) {
            double Tijk = T[idx(i, j, k, p.ny, p.nz)];
            double d2x = T[idx(i+1, j, k, p.ny, p.nz)] - 2*Tijk
```

```
15                          + T[idx(i-1, j, k, p.ny, p.nz)];
16              double d2y = T[idx(i, j+1, k, p.ny, p.nz)] - 2*Tijk
17                          + T[idx(i, j-1, k, p.ny, p.nz)];
18              double d2z = T[idx(i, j, k+1, p.ny, p.nz)] - 2*Tijk
19                          + T[idx(i, j, k-1, p.ny, p.nz)];
20              T_new[idx(i, j, k, p.ny, p.nz)] =
21                  Tijk + rx*d2x + ry*d2y + rz*d2z;
22          }
23      }
24      applyBC(T_new);       // parallel loops over 3 pairs of faces
25      std::swap(T, T_new);
26      time += dt;
27  }
```

Листинг А.3 — Средняя температура по объёму

```
1 double meanTemp(const Grid3D& T) {
2     double sum = 0.0;
3     const int n = (int)T.size();
4     #pragma omp parallel for reduction(+:sum) schedule(static)
5     for (int q = 0; q < n; ++q) sum += T[q];
6     return sum / (double)T.size();
7 }
```

Приложение Б

Сборка программного комплекса и запуск вычислительных экспериментов

В листинге Б.1 приведены команды сборки расчётного модуля для Linux и Windows, а в листинге Б.2 — команды запуска расчёта с заданным числом потоков, вычислительных экспериментов и автоматических тестов.

Листинг Б.1 — Сборка расчётного модуля

```
# Linux (GCC)
g++ -O2 -fopenmp -std=c++17 heat3d.cpp -o heat3d
# Windows (MinGW-w64, static linking)
x86_64-w64-mingw32-g++-posix -O2 -fopenmp -std=c++17 \
    -static heat3d.cpp -o heat3d.exe
# Linux: engine + GUI as a single executable (PyInstaller)
./build_linux.sh
```

Листинг Б.2 — Запуск расчёта, экспериментов и тестов

```
# simulation on a 100^3 grid with 4 threads
./heat3d --nx 100 --ny 100 --nz 100 --t_end 20000 --threads 4
# all experiments and figures of chapter 3
python nir_7sem/experiments.py
# speedup measurement for several grids
python report_plots.py --bench --threads 1 2 3 4 --nx 50 100 150
# automated tests of the parallel version
python -m pytest tests/integration/test_openmp.py -v
```

Ход выполнения практики

Задания и отметки о выполнении

1. Изучить технологию OpenMP и показатели эффективности параллельных программ для вычислительных систем с общей памятью.

Отметка о выполнении: _____ /0 — 100 баллов/

2. Разработать параллельную OpenMP-версию расчётного модуля программного комплекса HeatSim3D с управляемым числом потоков.

Отметка о выполнении: _____ /0 — 100 баллов/

3. Обеспечить сборку и работу программного комплекса в операционных системах Linux и Windows.

Отметка о выполнении: _____ /0 — 100 баллов/

4. Проверить корректность параллельной версии путём сравнения с аналитическим решением и с результатами последовательного расчёта.

Отметка о выполнении: _____ /0 — 100 баллов/

5. Исследовать ускорение и эффективность параллельной версии для расчётных сеток разного размера.

Отметка о выполнении: _____ /0 — 100 баллов/

6. Подготовить отчёт по практике в соответствии с требованиями нормоконтроля кафедры информационных систем и компьютерного моделирования.

Отметка о выполнении: _____ /0 — 100 баллов/

Отзыв о работе обучающегося

Отзыв пишет руководитель практики. Ниже — черновик для согласования.

В ходе практики Богунь М. В. разработал параллельную версию программного комплекса HeatSim3D для численного моделирования нестационарной теплопроводности с использованием технологии OpenMP. Студент провёл анализ зависимостей по данным в алгоритме явной разностной схемы, обеспечил работу программы в операционных системах Linux и Windows, выполнил проверку корректности по аналитическому решению и исследовал ускорение и эффективность параллельной версии на сетках разного размера.

Студент продемонстрировал умение планировать вычислительный эксперимент, количественно оценивать производительность программ и объяснять полученные результаты. Задачи практики выполнены в полном объёме.

Ответственный за		к.ф.-м.н., доцент каф. ИСКМ
организацию практики	_____ (подпись)	С. С. Храпов